The major difference with experience replay training is that we train with mini-batches of data when our replay list is full. We randomly select a subset of experiences from the replay, and we separate out the individual experience components into `state1_batch`, `reward_batch`, `action_batch`, and `state2_batch`. For example, `state1 _batch` is of dimensions `batch_size` × 64, or 100 × 64 in this case. And `reward_batch` is just a 100-length vector of integers. We follow the same training formula as we did earlier with fully online training, but now we're dealing with mini-batches. We use the tensor `gather` method to subset the `Q1` tensor (a 100 × 4 tensor) by the action indices so that we only select the Q values associated with actions that were actually chosen, resulting in a 100-length vector.

Notice that the target Q value, `Y = reward_batch + gamma * ((1 - done_batch) * torch.max(Q2,dim=1)[0])`, uses `done_batch` to set the right side to 0 if the game is done. Remember, if the game is over after taking an action, which we call a *terminal state*, there is no next state to take the maximum Q value on, so the target just becomes the reward, r_{t+1}. The `done` variable is a Boolean, but we can do arithmetic on it as if it were a 0 or 1 integer, so we just take `1 - done` so that if `done = True`, `1 - done = 0`, and it sets the right-side term to 0.

We trained for 5,000 epochs this time, since it's a more difficult game, but otherwise the Q-network model is the same as before. When we test the algorithm, it seems to play most of the games correctly. We wrote an additional testing script to see what percentage of games it wins out of 1,000 plays.

Listing 3.6. Testing the performance with experience replay

```
max_games = 1000
wins = 0
for i in range(max_games):
    win = test_model(model, mode='random', display=False)
    if win:
        wins += 1
win_perc = float(wins) / float(max_games)
print("Games played: {0}, # of wins: {1}".format(max_games,wins))
print("Win percentage: {}".format(win_perc))
```